GATE Complexity Analysis

Complete Solutions with Explanations

Q1: T(n) = T(n/2) + T(2n/5) + 7n, T(0) = 1

Answer: T(n) = Theta(n)

Using the Akra-Bazzi method: We solve (1/2)^p + (2/5)^p = 1. Since at p=1, the sum is 0.9 < 1, we get 0 < p < 1. The integral of f(u)/u^(p+1) from 1 to n gives Theta(n^(1-p)). Multiplying by n^p yields Theta(n). Alternatively, the non-recursive term 7n dominates because the sum of recursive subproblem sizes (1/2 + 2/5 = 0.9) < 1.

Q2: T(n) = aT(n/b) + f(n). Conditions for T(n) = Theta(n log n)

Answer: a = b and f(n) = Theta(n)

By the Master Theorem Case 2: If f(n) = Theta(n^(log_b a)), then T(n) = Theta(n^(log_b a) log n). For T(n) = Theta(n log n), we need log_b a = 1, which implies a = b. Then f(n) must be Theta(n^(log_b a)) = Theta(n).

Q3: Algorithm checks if array of size N is sorted by single pass comparing adjacent elements

Answer: Theta(N)

The algorithm makes exactly N-1 comparisons in the worst case and Omega(N) comparisons are necessary (must examine every adjacent pair). Thus, the tightest bound is Theta(N).

Q4: Asymptotic order of 1^3 + 2^3 + ... + n^3

Answer: Theta(n^4), O(n^5), Omega(n^3) are TRUE

Sum of cubes = [n(n+1)/2]^2 = n^2(n+1)^2 / 4 = (n^4 + 2n^3 + n^2)/4. Leading term is n^4/4, so the sum is Theta(n^4). Since n^4 = O(n^5) and n^4 = Omega(n^3), those options are also correct. Theta(n^5) is FALSE.

Q5: Code Analysis - fun1(int n)

Answer: q = Theta(n log log n)

Outer loop runs (n-1) times. First inner loop (j=n; j>1; j=j/2) runs log_2(n) times, so p = log_2(n). Second inner loop (k=1; k<p; k=k*2) runs log_2(p) = log_2(log_2(n)) times. Total: q = (n-1) * log_2(log_2(n)) = Theta(n log log n).

Q6: Nested loops - i=n; i>0; i/=2, inner j=0 to i

Answer: Theta(n)

Total work = n + n/2 + n/4 + n/8 + ... + 1 = n(1 + 1/2 + 1/4 + ...) = n * 2 = 2n. This geometric series converges to 2n, giving Theta(n).

Q7: Nested loops - i=1 to n, j=1 to i

Answer: Theta(n^2)

Total work = 1 + 2 + 3 + ... + n = n(n+1)/2 = (n^2 + n)/2. Leading term is n^2/2, giving Theta(n^2).

Q8: Recursion - fun(n/2) called twice

Answer: Theta(n)

Recurrence: T(n) = 2T(n/2) + O(1). By Master Theorem: a=2, b=2, log_b(a) = 1. f(n) = O(1) = O(n^0) where 0 < 1, so Case 1 applies: T(n) = Theta(n^log_b(a)) = Theta(n).

Q9: Fibonacci Pattern - fib(n-1) + fib(n-2)

Answer: Theta(phi^n) where phi = (1+sqrt(5))/2 approx 1.618, or O(2^n)

Recurrence: T(n) = T(n-1) + T(n-2) + O(1). The characteristic equation is x^2 = x + 1, with roots phi = (1+sqrt(5))/2 and psi = (1-sqrt(5))/2. Since |psi| < 1, the dominant term is phi^n. In GATE contexts, O(2^n) is often accepted as the upper bound, but the tight bound is Theta(phi^n).

Q10: Arrange in increasing order

Answer: n log n < n^(3/2) < n^(log_2 n) < 2^n

1. n log n vs n^(3/2): log n grows slower than n^(1/2), so n log n < n^(3/2).
2. n^(3/2) vs n^(log_2 n): For n > 4, log_2(n) > 2 > 3/2, so n^(log_2 n) dominates n^(3/2).
3. n^(log_2 n) vs 2^n: n^(log_2 n) = 2^((log_2 n)^2) is quasi-polynomial, which grows slower than any exponential 2^n.
Note: n^(log_2 n) is also written as 2^((log n)^2).

Q11: T(n) = T(n-1) + 1

Answer: Theta(n)

T(n) = T(n-1) + 1 = T(n-2) + 2 = ... = T(0) + n = Theta(n). This is linear - each level adds constant work 1.

Q12: T(n) = T(n-1) + n

Answer: Theta(n^2)

T(n) = T(n-1) + n = T(n-2) + (n-1) + n = ... = T(0) + 1 + 2 + ... + n = n(n+1)/2 = Theta(n^2).

Q13: T(n) = 2T(n-1) + 1

Answer: Theta(2^n)

T(n) = 2T(n-1) + 1 = 2[2T(n-2)+1] + 1 = 4T(n-2) + 3 = ... = 2^n * T(0) + (2^n - 1) = Theta(2^n). This is a geometric progression with doubling at each step.

Q14: T(n) = T(n/2) + 1

Answer: Theta(log n)

By Master Theorem: a=1, b=2, f(n)=1=Theta(n^0), log_b(a) = 0. Case 2 applies: T(n) = Theta(log n). By iteration: T(n) = T(n/2) + 1 = T(n/4) + 2 = ... = T(1) + log_2(n) = Theta(log n).

Q15: Nested loops - i=1; i<=n; i*=2, j=1 to n

Answer: Theta(n log n)

Outer loop runs log_2(n) times (i = 1, 2, 4, 8, ...). Inner loop runs n times for each outer iteration. Total = n * log_2(n) = Theta(n log n).